

Nama: Hans Malvin Djojo

<https://github.com/hansmalvin/LastMile>

1. Definition of Validation

Validation acts as a gatekeeper, checking that user data meets required rules before the system accepts it. Common examples include enforcing password lengths or blocking alphabetic characters in phone number fields. By catching mistakes at the source, validation keeps data clean, reduces technical glitches, and makes the interface more user-friendly.

2. Categories of Validation

Validation can generally be divided into two main types:

Client-Side Validation

Executed directly within the user's browser, this method offers real-time feedback. It identifies errors instantly, which creates a more responsive and seamless interaction by eliminating the need to wait for a page reload.

Server-Side Validation

This check happens on the back-end after the information is submitted. It acts as a vital security layer because it operates independently of the user's environment, ensuring that all data is strictly scrutinized before being stored or further processed.

3. Understanding Errors

Errors represent moments where software fails to execute an operation, often due to user mistakes or infrastructure issues. Because they are inevitable, the focus must be on managing them gracefully. Proper handling preserves system stability and provides the user with clear guidance rather than technical chaos.

4. Common Types of Errors

Input Validation Errors

These occur when submitted information fails to satisfy specific criteria, such as when a required field is left blank.

Authentication Errors

These happen when an individual is unable to prove their identity, typically due to providing the wrong login details.

Authorization Errors

These arise when a user attempts to reach a specific resource or area of the system that they are not cleared to access.

System Errors

These are triggered by internal infrastructure failures, such as server crashes or database malfunctions.

5. Communicating Errors Effectively

Error messages should be user-centric, offering specific details instead of technical jargon. A high-quality message identifies the problem and provides a solution. For example, specific instructions like "File size exceeds the 5MB limit" are far superior to vague notifications like "Upload failed."

6. Overview of HTTP Status Codes

HTTP status codes are used by servers to indicate the result of a request.

-2xx (Success): Indicates that the server successfully received and processed the request as intended.

-4xx (Client Errors): Signifies a problem with the request itself, typically due to unauthorized access or invalid data provided by the user.

-5xx (Server Errors): Marks an internal failure where the server was unable to complete an otherwise valid request.

These standardized codes offer developers a precise way to diagnose issues while shielding the end-user from complex or sensitive technical details.

7. Recommended Practices

Reliable apps don't just rely on frontend validation; they verify everything on the server.

Proper error handling is vital to keep sensitive system data hidden from the public. To balance technical needs with user experience, developers should keep detailed internal logs for fixing bugs while keeping user-facing messages straightforward and helpful.